

CSCI 585 — Master Cheatsheet *(with ready-to-write examples)*

Every concept comes with an example you can copy onto the exam. Green = example · Pink = the prof's exam hints.

SCOPE: Intro → Data Modeling → ER/EER → Relational Model & Algebra → Normalization → SQL & "More SQL" → Transactions (ACID). · Distributed/CAP = skim (see note). · **Excluded:** Tuesday's lecture & MongoDB/NoSQL.

READ FIRST How this professor writes the exam

- **"0% from the slides."** He tests what he says in class — understanding, not memorizing ("memorizing is overfitting").
- He shows a **diagram you've never seen** and asks you to reason about it.
- Format: **on paper, ~1 hr, mostly multiple-choice** + short written. He provides any reference sheet on exam day.
- **SQL:** questions *about* SQL (what it does / what's wrong / output) — he will **not** ask you to *write* SQL for a scenario.
- He rewards being able to **invent your own example** and **read an ER diagram cold** → that's why this sheet front-loads examples.

★ The professor's explicit exam hints

1. **CLOSURE** is asked almost every time — know *why* table-in/table-out enables chaining.
2. **Invent your own 1:1 / 1:M / M:N example.**
3. **Give a four-way (degree-4) participation example.**
4. **Read an unseen ER diagram:** "given a rectangle, what are all these ellipses?"
5. **DELETE vs DROP.**
6. **How to make a query/constraint FAIL.**
7. **LERP** (linear interpolation) has been asked before.
8. **Aggregate/computed columns** (an AVG is one value for the whole table).
9. Weak entities, cascade ordering, the 8 relational-algebra operators — all fair game.

Examples Bank — copy these straight onto the exam

Connectivity (he may ask you to invent one)

TYPE	EXAMPLE
1:1	one EMPLOYEE manages one DEPARTMENT (and a dept has one manager); a PERSON ↔ one PASSPORT
1:M	one PROFESSOR teaches many CLASSES; one PAINTER paints many PAINTINGS; one LOAN has many PAYMENTS
M:N	many STUDENTS enroll in many COURSES; many AUTHORS write many BOOKS

Relationship DEGREE

DEGREE	EXAMPLE
1 — unary/recursive	EMPLOYEE <i>manages</i> EMPLOYEE; COURSE <i>is prerequisite of</i> COURSE
2 — binary	PROFESSOR <i>teaches</i> CLASS (the default in all his examples)
3 — ternary	DOCTOR <i>prescribes</i> DRUG <i>to</i> PATIENT
4 — quaternary	DOCTOR prescribes DRUG to PATIENT at a PHARMACY — one relationship linking 4 entities (Doctor, Patient, Drug, Pharmacy). Another: SUPPLIER supplies PART for PROJECT under CONTRACT.
5 — quinary	SUPPLIER ships PART to WAREHOUSE for PROJECT via CARRIER — one "shipment" relationship linking 5 entities.

Why degree-4 can't just be split into binaries: the fact "this doctor gave this drug to this patient at this pharmacy" needs all four together to be meaningful.

Attribute types

- **Composite:** NAME →(first, last); ADDRESS →(street, city, zip)
- **Multivalued:** a PERSON's PHONE numbers; a CAR's COLOR s
- **Derived:** AGE from DOB; LINE_TOTAL = qty × price

Weak entity

DEPENDENT needs an EMPLOYEE to exist (its PK = emp_id + dep_name). Also: invoice LINE needs an INVOICE ; APARTMENT needs a BUILDING .

Bridge (composite) entity — resolves M:N

ENROLLMENT sits between STUDENT and CLASS ; PK = (stu_id, class_id). Also ORDER_LINE between ORDER and PRODUCT.

EER subtypes

- **Disjoint (d):** ACCOUNT → {CHECKING, SAVINGS} (an account is one or the other)
- **Overlapping (o):** PERSON → {EMPLOYEE, STUDENT} (a student-worker is both)
- **Total:** every EMPLOYEE must be {FACULTY or STAFF}.
Partial: a PERSON may be neither a student nor an employee.

LERP — formula + example

$\text{lerp}(a, b, t) = a + t \cdot (b - a)$, $t \in [0, 1]$. E.g. $\text{lerp}(10, 20, 0.5) = 15$; $t=0 \rightarrow a$, $t=1 \rightarrow b$. Used to interpolate a value between two known ones.

1 DB Fundamentals

Data ≠ Information. Data = raw facts; information = processed/contextualized data useful for decisions.

EXAMPLE

42 is data; "the patient's temperature is 42°C — dangerously high" is information.

DBMS = software **go-between**: User ↔ DBMS ↔ data .
Manages access, structure, security, concurrency, recovery.

DBMS VS FILE SYSTEMS

- Removes **structural & data dependence**; reduces **redundancy**/anomalies and **"islands of information"** (siloes, inconsistent files).
- Adds sharing, security, integrity, backup, a query language.

EXAMPLE — STRUCTURAL DEPENDENCE

In a file system, adding a "phone2" field can break every program that reads the file. A DBMS hides that from apps.

Categorize DBs by: #users, location (centralized/distributed/cloud), use (OLTP operational vs OLAP analytical/warehouse), structure (relational/NoSQL).

2 Data Modeling & Abstraction

A **model** = abstraction of the real thing (like engineering simulation: keep what matters).

Evolution

Hierarchical (tree, single-parent) → **Network** (graph, many-parent) → **Relational** (tables) → **ER** → **OO / Object-Relational** → **NoSQL** → **Vector** (ML embeddings).

EXAMPLE

Hierarchical = a company org chart (each employee one boss). Network = a flight map (a city reachable from many cities). Relational = a spreadsheet of tables.

Four abstraction layers

LEVEL	WHO	CAPTURES
External	User	per-stakeholder views
Conceptual	Designer	unified, DBMS-independent (ER here)
Internal	Programmer	DBMS-specific logical schema
Physical	Implementer	files, blocks, indexes

3 ER Diagrams

Entity = rectangle (→ table). **Attribute** = ellipse (→ column). **Relationship** = diamond (verb).

DIAGRAM-READING HINT

"Given a rectangle, what are the ellipses?" → rectangle = entity/table; ellipses = its attributes/columns.

The 4 properties of any relationship

PROPERTY	MEANING	EXAMPLE
Connectivity	type: 1:1, 1:M, M:N	prof→classes = 1:M
Cardinality	(min,max) numbers inside	(1,4)
Participation	optional(min 0) / mandatory(min 1)	prof may teach 0 → optional
Degree	# entities involved	binary = 2

READING CARDINALITY (1,4)

Forward: a professor teaches **min 1, max 4** classes.
Backward (other side): each class is taught by **min 1, max 1** professor.

Weak entity (BOTH conditions)

(1) existence-dependent on an owner, AND (2) identifying relationship (its PK *includes* the owner's key). Double rectangle + double diamond.

EXAMPLE

DEPENDENT can't exist without **EMPLOYEE** ; PK = (emp_id, dep_name). Test: "can I exist apart from you?" → no.

Bridge entity

Turns one **M:N** into two **1:M**; PK = composite of the two FKs.

EXAMPLE

ENROLLMENT (stu_id, class_id, grade) between **STUDENT** and **CLASS**.

3c Extended ER (EER)

Adds **inheritance**: a **supertype** + specialized **subtypes** ("is-a"). Columns flow down like public single inheritance.

EXAMPLE

EMPLOYEE (super) → PILOT , MECHANIC ,
ACCOUNTANT (subs). Each sub inherits emp_id/name
and adds its own columns (e.g. pilot license #).

DISJOINTNESS	COMPLETENESS
Disjoint (d) : at most one subtype <i>ACCOUNT → {CHECKING, SAVINGS}</i>	Total : must be in a subtype <i>every EMPLOYEE is faculty or staff</i>
Overlapping (o) : can be several <i>PERSON → {EMPLOYEE, STUDENT}</i>	Partial : may be in none <i>a PERSON may be neither</i>

Specialization = top-down split; **Generalization** = bottom-up merge. **Lattice** = subtype with >1 supertype. **Union (category)** = a subtype related to several distinct supertypes (e.g. OWNER = PERSON *or* COMPANY).

4 Relational Model & Keys

Relation=table, **tuple**=row, **attribute**=column, **domain**=allowed value range.

Functional dependency (FD)

$X \rightarrow Y$: **X determines Y** (each $X \rightarrow$ exactly one Y). X = determinant.

EXAMPLE

STU_ID → STU_NAME (an ID gives exactly one name).
(PROJ_NUM, EMP_NUM) → HOURS .

Key types

KEY	MEANING / EXAMPLE
Superkey	any unique attr set: (stu_id, name)
Candidate	minimal superkey: stu_id, or email
Primary	chosen candidate; unique + NOT NULL
Composite	≥2 cols: (proj_num, emp_num)
Foreign	v_code in PRODUCT → PK of VENDOR
Secondary	for retrieval, not unique: last_name

Integrity rules

- **Entity**: PK unique & not null.
- **Referential**: FK matches an existing PK *or* is null.

EXAMPLE — REFERENTIAL

You can't insert a PRODUCT with v_code=99 if no VENDOR has v_code 99.

Nulls are ambiguous → avoid where possible.

4b Relational Algebra & ★ CLOSURE

HIGH-FREQUENCY TOPIC

Closure = every operator takes **table(s) in** → **table out** (same type in/out), so operators **chain** (data flow). Output can be a valid **0-row table**.

EXAMPLE OF CHAINING (CLOSURE)

SELECT high-GPA USC students → SELECT high-GPA UCLA students → UNION them → SELECT only CS majors. Each step's output is a table feeding the next.

Codd's 8 operators

OP	DOES	MINI-EXAMPLE
SELECT σ	subset of rows	students WHERE gpa ≥ 3.5
PROJECT π	subset of cols	just (name, gpa)
UNION	rows of A + B	USC $\cup$ UCLA students
INTERSECT	rows in both	students at both schools
DIFFERENCE	$A - B$	USC students not at UCLA
PRODUCT $\times$	Cartesian product	every student $\times$ every course
JOIN	combine on match	PRODUCT $\bowtie$ VENDOR on v_code
DIVIDE	matches ALL of B	students taking <i>every</i> CS course

UNION/INTERSECT/DIFFERENCE need **union-compatible** tables.

SET THEORY LINK

These three are literally **set operations** on rows:

UNION = $A \cup B$ (everything in either), **INTERSECT** = $A \cap B$ (in both), **DIFFERENCE** = $A - B$ (in A, not B).

Picture the Venn diagram.

JOIN FLAVORS

Inner/Natural=only matches. **Left/Right/Full outer**=keep unmatched rows from left/right/both (NULL-filled).

5 Normalization — full worked example (his construction-company case)

Goal: kill redundancy → kill anomalies. Done early, **one table at a time**.

THE 3 ANOMALIES (MEMORIZE VIA EXAMPLE)

- **Insert:** can't add a new PROJECT until an employee is assigned to it.
- **Update:** an employee's job rate is stored in many rows — change one, miss another → inconsistent.
- **Delete:** deleting the last row for a project also erases the project's name.

THE TWO BAD DEPENDENCIES

- **Partial:** non-key depends on *part* of a composite PK.
PROJ_NUM → PROJ_NAME (PROJ_NUM is only half the PK).
- **Transitive:** non-key → non-key. *EMP_NUM → JOB_CLASS → CHG_HOUR.*

The ladder

FORM	ELIMINATE	PARTIAL / TRANSITIVE LEFT?
1NF	repeating groups (no blanks); set PK	y / y
2NF	partial dependencies	n / y
3NF	transitive dependencies	n / n
BCNF	every determinant is a candidate key	—

Starting table (unnormalized report): columns are

PROJ_NUM, PROJ_NAME, EMP_NUM, EMP_NAME, JOB_CLASS, CHG_HOUR, HOURS

1NF

Fill blanks (no repeating groups), pick PK = **(PROJ_NUM, EMP_NUM)**. Still has partial + transitive deps.

2NF — SPLIT OUT PARTIAL DEPS

PROJECT(PROJ_NUM, PROJ_NAME)

EMPLOYEE(EMP_NUM, EMP_NAME, JOB_CLASS, CHG_HOUR)

ASSIGN(PROJ_NUM, EMP_NUM, HOURS)

3NF — SPLIT OUT TRANSITIVE DEP

JOB_CLASS → CHG_HOUR was transitive, so promote it:

JOB(JOB_CLASS, CHG_HOUR)

EMPLOYEE(EMP_NUM, EMP_NAME, JOB_CLASS)

(JOB_CLASS now an FK)

Rule of thumb he used: "for each problem dependency, create a new relation."
Top of a dependency diagram = full (good); bottom = partial/transitive (bad).
The end goal is **full functional dependence**: every non-key attribute depends on the whole key, nothing less.

DENORMALIZATION

Intentionally re-add redundancy for read speed (fewer joins)
— common in reports/warehouses.

6 SQL — Core (questions about SQL)

SQL is **declarative** (what, not how). DDL:
CREATE/ALTER/DROP · DML:
INSERT/SELECT/UPDATE/DELETE · TCL:
COMMIT/ROLLBACK.

HINT — DELETE VS DROP

Both wipe rows. DELETE FROM product; **keeps the table** (columns remain). DROP TABLE product; **removes the whole table** (must CREATE again).

```
SELECT cols FROM t
WHERE row-condition
GROUP BY c HAVING group-condition
ORDER BY c [ASC|DESC];
```

EXAMPLES PER PIECE

- **WHERE:** WHERE p_price > 50 AND v_code = 21
- **LIKE:** WHERE name LIKE 'A%' (starts with A); '_o%'
2nd letter o
- **BETWEEN / IN:** price BETWEEN 10 AND 20 ; v_code
IN (21,24)
- **IS NULL:** WHERE v_code IS NULL
- **Computed col (feature eng.):** SELECT p_qoh*p_price
AS stock_value

- **CASCADE:** ON DELETE CASCADE — deleting a
VENDOR auto-deletes its PRODUCTS.

HINT — MAKE IT FAIL

WHERE pid=3 AND pid=5 → no single row can be
both → returns 0 rows. (PID is a PK, so impossible.)

6b SQL — Grouping & Aggregates

Aggregates: COUNT, SUM, AVG, MIN, MAX — collapse rows → one value.

EXAMPLE

```
SELECT AVG(p_price) FROM product; → one
number for the whole table.
```

GROUP BY = "categorize by"

EXAMPLE

```
SELECT v_code, AVG(p_price) FROM product GROUP
BY v_code; → one avg per vendor.
```

- GROUP BY without an aggregate = meaningless.
- **WHERE** filters rows *before* grouping; **HAVING** filters groups *after*.

WHERE VS HAVING

```
... WHERE p_price>10 GROUP BY v_code HAVING
COUNT(*)>3;
```

DISTINCT drops dupes · ORDER BY a, b multi-sort ·
bulk insert via INSERT ... SELECT

7 "More SQL"

Subqueries (query in a query; runs inner-first, like a call stack)

- **WHERE subquery:** WHERE price > (SELECT AVG(price) FROM product)
- **FROM subquery:** treat a query's result as a table.
- **Correlated:** inner query re-runs *for each outer row* (references it). *e.g. products priced above their own vendor's average.*
- **ALL / ANY:** > ALL(...) => the max; > ANY(...) => the min.

Set ops

UNION , INTERSECT , EXCEPT/MINUS (need union-compatible cols).

Views / sequences / programmable

- **VIEW** = saved virtual query (a window on data, not stored rows).
- **Sequence** = auto-numbering; NEXTVAL = current then ++.
- **PL/SQL** = procedural (vars, IF, loops) in DECLARE/BEGIN/END.
- **Trigger** = auto-fires BEFORE/AFTER INSERT/UPDATE/DELETE.
- **Stored procedure** (does an action) vs **stored function** (returns a value).

TRIGGER EXAMPLE

```
AFTER INSERT on SALE → auto-decrement
PRODUCT.p_qoh.
```

8 Transactions — ★ ACID

A **transaction** = one atomic unit of work (reads/writes), moving the DB consistent→consistent.

LETTER	GUARANTEE	EXAMPLE
Atomicity	all-or-nothing	bank transfer: if debit succeeds but credit fails, roll back both
Consistency	rules never broken	total money before = total after
Isolation	acts as if alone	two people buying the last seat don't both succeed
Durability	survives crashes	once committed, a power outage can't undo it

Textbook adds S (serializable) → he folds that into Isolation, not a 5th property.

Commit / Rollback

- COMMIT = make permanent (auto on clean finish).
- ROLLBACK = discard (auto on crash).

Concurrency

- Serial** = one-at-a-time. **Serializable** = interleaved but same result as *some* serial order.
- Locking** enforces isolation. **2-Phase Locking**: grow (acquire) then shrink (release) — never acquire after releasing.

LOST-UPDATE EXAMPLE

Two transactions read qoh=10, both subtract 1, both write 9 → should be 8. Locking prevents this.

Locks (managed by the lock manager — the "7-Eleven key" process)

LOCK	RULE
Shared / read	many transactions can read at once
Exclusive / write	only ONE; all read locks revoked first

Pessimistic locking = always lock (assume conflict).

Optimistic = no locks, used when conflicts are impossible.

OPTIMISTIC EXAMPLE (HIS)

A DB of all Monet paintings — a museum can add a row without locking; no one else is editing.

★ Deadlock (a "no isolation" hazard)

Two transactions each hold a lock the other needs → both wait forever (circular wait / "deadly embrace").

IDENTIFYING A DEADLOCK

Build a **wait-for graph** (node per transaction, arrow $T_i \rightarrow T_j$ if T_i waits for a lock T_j holds). A **cycle = deadlock**.

- Deadlock prevention** — structure locking so a cycle can never form.

- Deadlock detection/avoidance** — let it happen, detect the cycle, then abort/roll back one transaction (the "victim").

9 Distributed DBs + CAP (skim)

Scope note: *He waffled on whether this is on the midterm (later leaned toward pushing it to Tuesday = excluded). Skim only; ACID above is the safe bet.*

Distributed processing = many CPUs, one DB.

Distributed database = data itself spread across sites.

Fully distributed = both (MP/MD). Variants: SP/SD, MP/SD, MP/MD (no SP/MD).

- TP** requests data; **DP** manages stored data.
- Fragmentation**: split a table (horizontal=rows, vertical=cols). **Replication**: copies at sites.
- Transparency**: user thinks it's one local DB (via Distributed Data Dictionary).
- 2-Phase Commit (2PC)**: Phase 1 = coordinator asks "ready?" (vote); Phase 2 = all yes → commit, else abort everywhere.

CAP (BREWER)

Pick **2 of 3**: Consistency, Availability, Partition-tolerance. Partitions happen, so real systems trade C vs A during one.

CAP EXAMPLE

Banking favors **C** (refuse rather than show wrong balance); a social feed favors **A** (show slightly stale likes rather than error).

10 Agentic AI & Text-to-SQL

A recurring theme of his: you increasingly **prompt** for queries instead of hand-writing SQL.

- Text-to-SQL**: natural-language request → generated SQL query. (The homeworks use this.) "Show vendors in CA with >5 products" → the model writes the SELECT.
- Agentic AI**: an agent can **spin up a database, use it, and tear it down** in seconds, and generate/run SQL itself — so "SQL is going away" as something humans type by hand.
- Vector embeddings**: text/audio/video → numeric vectors stored in a **vector DB**; similarity search powers agent "memory"/retrieval. (This is the modern "vector" data model from §2.)

Likely tested conceptually (what text-to-SQL is, why a vector DB exists, how agents use DBs) — not by asking you to write a prompt.

11 Confirmed from his newest papers

Closure beyond tables (he tests this!)

Closure = output type = input type → chaining. He asks it on **non-DB** things: determinant **breaks** closure on square matrices (→ scalar); averaging/blur or add-with-clip **preserves** closure on colors [r,g,b]; grayscale breaks it.

Set theory / De Morgan

$\neg(A \cup B) = \neg A \cap \neg B$ and $\neg(A \cap B) = \neg A \cup \neg B$. To verify: define the universe U, build each side explicitly, show they're equal.

JSON fixes a relational drawback

A cell can't hold **multivalued/nested** data. JSON uses arrays/nesting: "degree": ["BS", "MS", "PhD"] .

2PL vs 2PC

Both: 2-phase, keep correctness with concurrency. **2PL** = locking → serializability, single node. **2PC** = commit protocol → atomic commit across nodes.

SQL = code

FROM+JOIN $\equiv$ nested for loops; WHERE $\equiv$ if filter.
(loop → if → collect)

PK = determination

A PK "points to" non-prime attrs via **functional dependency**. Single-table fault = **entity-integrity** violation (null/dup PK); cross-table = **referential-integrity** violation (FK → missing row).

Other one-liners

Too much normalization → slow (excessive joins);
remedy = denormalize. **Special-purpose DBs** (molecule, fingerprint, stocks): special viewer/matcher interface, non-row queries.

Built from CSCI 585 decks (Lec 1–7) + transcripts (May 20 – Jun 4, 2026). Construction-company example = the one from his Normalization slides. Understand > memorize — that's how the exam is written. Good luck Wednesday.